

Reporte Proyecto 1: Modelado e Implementación de un Chat Cliente-Servidor

Alvarado Trujillo Isaac Armando

29 de septiembre de 2026

1. Resumen

Proyecto desarrollado para la materia de Modelado y Programación. Consiste de un sistema de mensajería instantánea concurrente, con la arquitectura Cliente-Servidor mediante sockets TCP/IP y usando un protocolo de comunicación estructurado en formato JSON.

2. Introducción

2.1. Objetivo del Proyecto

Desarrollar, modelar y validar una solución cliente-servidor para mensajería que implemente la especificación formal del protocolo basado en JSON, garantizando una concurrencia segura, administrar salas privadas dinámicas mediante invitaciones, control de presencia de usuarios y manejo de fallos.

2.2. Alcance

El sistema contempla lo siguiente:

- **Autenticación de sesión:** Identificación obligatoria de usuarios con nombres únicos de hasta 8 caracteres antes de poder ejecutar cualquier acción en el servidor.
- **Control de presencia:** Tres estados de usuario (**ACTIVE**, **AWAY**, **BUSY**), notificando inmediatamente los cambios al resto de la comunidad conectada.
- **Intercambio de mensajes:** Envío de mensajes globales a todos los usuarios conectados y mensajería directa entre usuarios registrados.
- **Gestión de salas de salas de conversación:** Creación de salas privadas con nombres de hasta 16 caracteres, mecanismo de invitación selectiva, verificación de membresía antes del ingreso, intercambio de mensajes exclusivos dentro del canal y autodestrucción inmediata de la sala cuando su último miembro activo la abandona.
- **Desconexión ordenada:** Liberación de recursos en memoria y aviso de desconexión al resto de clientes.

2.3. Posibles problemas enfrentados y facilidades encontradas

Durante el desarrollo se identificó algunas complejidades técnicas:

- **Gestión de memoria y concurrencia:** El servidor maneja punteros compartidos entre hilos para representar a los usuarios y sus participaciones en salas. La eliminación prematura de un usuario mientras otro hilo itera sobre la lista de la sala requirió un esquema de exclusión mutua mediante dos mutex independientes: `mutexUsuarios` y `mutexSalas`.
- **Operabilidad entre plataformas:** Asegurar que la serialización de C# generara cadenas UTF-8 idénticas a las esperadas por cJSON en C.

Las facilidades destacan:

- El uso de la biblioteca de macros `uthash`, que permite dotar a cualquier estructura en C de capacidades de tabla hash sin necesidad de escribir algoritmos de dispersión y rehashing manual.
- La biblioteca cJSON, cuyo árbol jerárquico simplificó la extracción tipada y validación de campos.
- Las facilidades de .NET 9 en C# (`TcpClient`, `StreamReader.ReadLine()` y `JsonDocument`), que abstraen el framing por saltos de línea y la deserialización no tipada.

3. Análisis y Diseño (Modelado)

Para cumplir con los requerimientos, la arquitectura del servidor se diseñó bajo los siguientes enfoques:

3.1. Patrones de Diseño y Principios Arquitectónicos

El sistema implementa varios patrones reconocidos de la ingeniería de software:

1. **Patrón Despachador / Controlador (Dispatcher / Front Controller):** Implementado en `controlador.c` a través de la función `EnrutarMensaje()`. El controlador actúa como punto único de entrada para todas las peticiones entrantes de un cliente, valida la sintaxis JSON, comprueba el estado de autenticación y deriva el flujo hacia el manejador de negocio específico.
2. **Patrón Observador / Publicador-Suscriptor (Pub-Sub):** El servidor funge como un broker de mensajería. En las difusiones públicas (`TransmitirMensajePublico`) o de salas (`TransmitirTextoSala`), el canal actúa como sujeto o tópico, y los sockets conectados son los observadores que reciben la notificación sin acoplamiento entre sí.
3. **Patrón Monitor / Región Crítica:** En el módulo `estado.c`, el acceso a las estructuras dinámicas `listaUsuarios` y `listaSalas` está encapsulado detrás de funciones que adquieren y liberan rigurosamente los cerrojos `mutexUsuarios` y `mutexSalas`, garantizando la atomicidad e invariabilidad de los datos.
4. **Separación en Capas (Separation of Concerns):**
 - *Capa de Red (red.c):* Responsable exclusiva de la codificación y emisión física de bytes por los sockets.

- *Capa de Negocio y Control (controlador.c)*: Lógica de aplicación, enrutamiento y validaciones.
- *Capa de Datos y Estado (estado.c)*: Manipulación pura de las estructuras en memoria volátil.

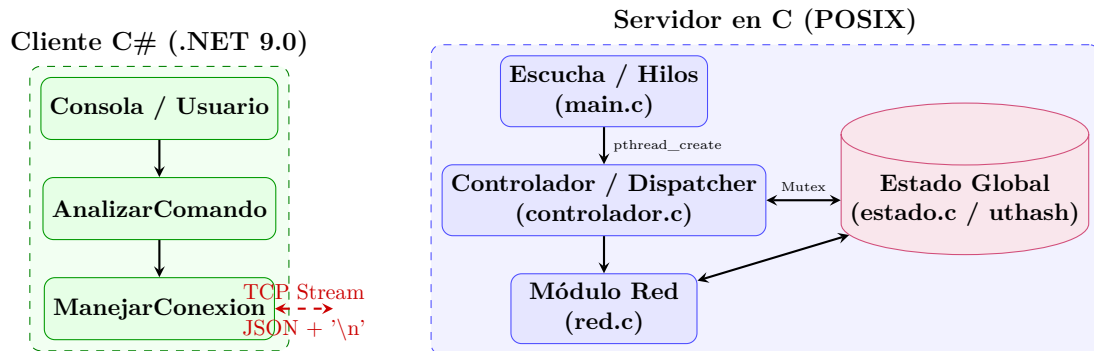


Figura 1: Diagrama de arquitectura del sistema de chat distribuido.

3.2. Modelado de Estructuras de Datos y clases

3.2.1. Estructuras en el Servidor (C)

- `struct Usuario`: Identificador `username[9]`, descriptor `socket`, estado `status[10]` y handle `UT_hash_handle hh`.
- `struct MiembroSala`: Asociación de usuario a sala con nombre y puntero a `Usuario`.
- `struct Sala`: Nombre `roomname[17]`, tablas hash internas (`activos` e `invitados`) y handle `hh`.
- `struct EstadoGlobal`: Tablas raíz (`listaUsuarios` y `listaSalas`) protegidas por `mutexUsuarios` y `mutexSalas`.
- `struct Cliente`: Contexto por hilo con `socket` privado, referencia al estado global y credencial validada.

3.2.2. Clases en el Cliente (C#)

- `Program`: Punto de entrada que procesa argumentos CLI (`ip` y `puerto`) e invoca `Chat`.
- `Chat`: Orquestador del ciclo de vida. Administra autenticación inicial y bifurca hilo receptor.
- `ManejarConexion`: Abstracción del socket TCP mediante `TcpClient`, `StreamReader` y `StreamWriter`.
- `AnalizarComando`: Parser bidireccional de comandos a JSON y de eventos JSON a texto coloreado.

3.3. Especificación del Protocolo de Comunicación

El protocolo define una gramática estricta en JSON donde todo mensaje finaliza invariablemente en `\n`. En las Tablas 1 y 2 se resumen las interacciones fundamentales del sistema.

3.4. Mapeo de Comandos de Interfaz en C#

En la Tabla 3 se detalla la traducción interna que efectúa la clase `AnalizarComando` desde la entrada textual de consola hacia los mensajes JSON del protocolo.

Comando (Tipo)	Parámetros Clave	Respuesta	Servi- dor	Efecto / Propósito
IDENTIFY	username: string	RESPONSE	/	Registra el usuario si no existe (máx 8 car.).
STATUS	status: string	NEW_USER		Cambia estado a ACTIVE, AWAY o BUSY.
USERS	<i>Ninguno</i>	NEW_STATUS (broadcast)		Retorna catálogo global de usuarios y estados.
TEXT	username, text	USER_LIST		
PUBLIC_TEXT	text: string	Directo	o	Mensaje privado al destinatario indicado.
NEW_ROOM	roomname: string	NO_SUCH_USER		Difusión general a todos los usuarios.
INVITE	roomname, username	PUBLIC_TEXT_FROM		Crea una nueva sala (máx 16 car.) y une al creador.
JOIN_ROOM	roomname: string	RESPONSE	/	Invita a usuarios si el emisor ya es miembro.
ROOM_USERS	roomname: string	JOINED_ROOM		Se une a una sala habiendo sido invitado.
ROOM_TEXT	roomname, text	ROOM_USER_LIST		Consulta lista de miembros de la sala.
LEAVE_ROOM	roomname: string	ROOM_TEXT_FROM		Envía mensaje grupal exclusivo en la sala.
DISCONNECT	<i>Ninguno</i>	LEFT_ROOM	/	Abandona sala; si queda vacía, se destruye.
		RESPONSE		Cierra sesión, abandona salas y libera socket.
		DISCONNECTED		

Cuadro 1: Mensajes enviados desde el cliente hacia el servidor.

Mensaje Servidor	Parámetros Clave	Significado y Destinatario
RESPONSE	operation, result, [extra]	Resultado de operación (SUCCESS, USER_ALREADY_EXISTS, etc.).
NEW_USER	username	Notificación global a otros usuarios ante nueva conexión.
NEW_STATUS	username, status	Notificación global ante cambio de estado de un participante.
USER_LIST	users: {user: status}	Diccionario con todos los usuarios conectados y sus estados.
TEXT_FROM	username, text	Entrega de mensaje privado recibido de otro usuario.
PUBLIC_TEXT_FROM	username, text	Entrega de mensaje de difusión general recibido.
INVITATION	username, roomname	Notificación al usuario de que ha sido invitado a unirse a una sala.
JOINED_ROOM	roomname, username	Notificación a miembros de sala sobre la incorporación de alguien.
ROOM_USER_LIST	roomname, users: {...}	Catálogo de integrantes activos en una sala específica.
ROOM_TEXT_FROM	roomname, username, text	Mensaje grupal entregado a los miembros de la sala.
LEFT_ROOM	roomname, username	Aviso a miembros de sala sobre el abandono de un usuario.
DISCONNECTED	username	Aviso global de que un usuario ha abandonado el servidor.

Cuadro 2: Mensajes y notificaciones emitidos por el servidor hacia los clientes.

Comando CLI	Sintaxis de Entrada	Acción y Tipo de Mensaje JSON
/estado	/estado <AWAY BUSY ACTIVE>	Genera trama STATUS con nuevo estatus.
/usuarios	/usuarios	Genera trama USERS para solicitar el listado.
/privado	/privado <user><texto>	Genera trama TEXT con destinatario y contenido.
/publico	/publico <texto>	Genera trama PUBLIC_TEXT para difusión general.
/nuevasala	/nuevasala <nombre>	Genera trama NEW_ROOM para fundar un canal.
/invitar	/invitar <sala><u1>[u2...]	Genera una o múltiples tramas INVITE.
/salausuarios	/salausuarios <sala>	Genera trama ROOM_USERS para ver miembros.
/mensajesala	/mensajesala <sala><texto>	Genera trama ROOM_TEXT para charla grupal.
/irse	/irse <sala>	Genera trama LEAVE_ROOM para abandonar el canal.
/cerrar	/cerrar	Genera trama DISCONNECT para cerrar sesión.
/ayuda	/ayuda	Muestra menú contextual en consola (sin enviar red).

Cuadro 3: Mapeo de comandos del cliente de consola a primitivas del protocolo.

3.5. Diagramas de Comportamiento

3.5.1. Diagrama de Secuencia: Autenticación y Envío de Mensaje Privado

La Figura 2 ilustra la secuencia de handshake y transmisión de mensajes privados entre dos clientes coordinados por el servidor.

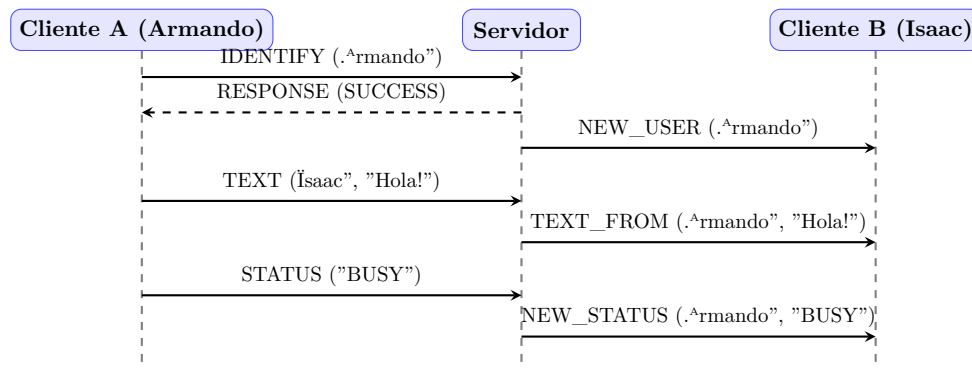


Figura 2: Diagrama de secuencia: Autenticación, difusión y mensajería privada.

3.5.2. Diagrama de Estados: Ciclo de Vida del Usuario y de las Salas

En la Figura 3 se representan las transiciones de estado de un usuario dentro de la sesión, así como el ciclo de vida de una sala desde su creación hasta su eliminación.

4. Implementación

4.1. Tecnologías y Entornos de Desarrollo

La implementación se hizo garantizando la máxima compatibilidad entre C, C# y Latex:

■ Servidor en C:

- *Sockets POSIX*: Llamadas del sistema `socket()`, `setsockopt(SO_REUSEADDR)`, `bind()`, `listen()`, `accept()`, `recv()` y `send()`.

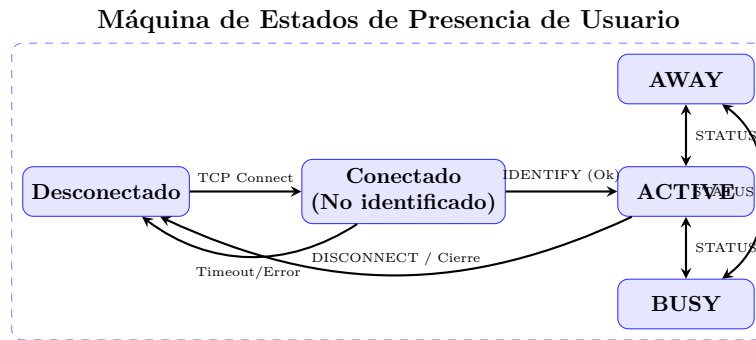


Figura 3: Máquina de estados de un usuario en el servidor.

- *Pthreads*: Creación de hilos independientes por cliente con `pthread_create()` en modo detached (`pthread_detach()`) para la recolección automática de recursos al finalizar.
- *uthash*: Biblioteca de un solo archivo de cabecera (`uthash.h`) que implementa estructuras hash en C usando macros de preprocesador.
- *cJSON*: Parser de JSON en C puro utilizado para construir y parsear los payloads JSON de manera dinámica.

■ Cliente en C# (.NET 9.0):

- *System.Net.Sockets*: Clase de alto nivel `TcpClient` que gestiona la conexión TCP subyacente.
- *System.IO*: Abstracción mediante `StreamReader` y `StreamWriter` con `AutoFlush = true`, asegurando que cada línea escrita se vacíe instantáneamente en el socket hacia la red.
- *System.Text.Json*: Procesamiento de documentos JSON mediante `JsonDocument` y serialización con `JsonSerializer`.
- *System.Threading*: Hilo secundario (`Thread`) asignado a la ejecución continua del método `EscucharMensajes()`.

■ Herramientas de Construcción:

- **GNU Make**: Automatización de compilación, ejecución de pruebas, limpieza y generación de reporte.
- **GCC**: Compilador GNU para C con banderas de optimización y depuración (`-Wall -Wextra -pthread`).
- **Tectonic**: Motor de composición de documentos XeTeX moderno que resuelve paquetes LaTeX de manera automatizada.

4.2. Lectura a trozos y Manejo del Buffer TCP

Uno de los componentes del servidor radica en la función `IniciarLectura()` ubicada en `controlador.c`. Debido a que TCP no entrega mensajes completos predefinidos, la implementación emplea un buffer lineal de hasta 1 MB (`MAX_MSG_SIZE`) con lecturas progresivas de fragmentos (`CHUNK_SIZE = 4096` bytes).

El algoritmo de lectura implementado opera de la siguiente forma:

1. Se reciben bytes con `recv()` y se agregan al final de la memoria ocupada (`buffer_len += n_recv`).
2. Se inspecciona el buffer en busca del carácter delimitador `\n` mediante la función de biblioteca de alta velocidad `memchr()`.
3. Si se encuentra un salto de línea:
 - Se calcula la longitud del mensaje $L = nl - buffer$.
 - Se reserva memoria temporal y se copia la cadena terminada en `\0`.
 - Si el mensaje no está vacío ni es un retorno de carro residual (`\r`), se transfiere a `EnrutarMensaje()`.
 - Los bytes posteriores al salto de línea se desplazan al inicio del buffer mediante `memmove(buffer, nl + 1, restantes)`, actualizando `buffer_len`.
4. Se repite el ciclo interno hasta procesar todas las tramas completas acumuladas en el buffer antes de volver a bloquearse en `recv()`.

4.3. Sincronización y Prevención

Como el uso de múltiples hilos de clientes compiten por modificar o leer las listas de usuarios y salas, en `estado.c` se optó por el bloqueo granular:

- **mutexUsuarios:** Protege exclusivamente las operaciones sobre `listaUsuarios` (registro de usuario, validación de existencia, consulta de socket y actualización de estado).
- **mutexSalas:** Protege las operaciones sobre `listaSalas`, incluyendo las tablas anidadas de miembros `activos` e `invitados` de cada sala.

4.4. Requerimientos del Sistema

Para la compilación y ejecución exitosa del proyecto, el entorno debe disponer de los siguientes elementos:

- Sistema Operativo: GNU/Linux (probado en distribuciones Linux Mint y Cachy Os).
- Compilador: `gcc` versión 9.0 o superior con soporte de POSIX Threads.
- Plataforma .NET: SDK de .NET 9.0 (`dotnet-sdk-9.0`).
- Herramienta de compilación: `make` (GNU Make).
- Motor de Reportes: `tectonic` versión 0.15 para la generación del PDF.

4.5. Instrucciones de Compilación y Ejecución

La raíz del proyecto incluye un archivo `Makefile` que automatiza todas las tareas operativas. En la Tabla 4 se resumen los comandos disponibles.

Comando Make	Acción Ejecutada	Descripción
make all	servidor cliente reporte	Compila ambos componentes y genera el reporte PDF.
make servidor	Compila con gcc el servidor	Produce el binario ejecutable <code>Servidor/servidor</code> .
make abrir-servidor	<code>./Servidor/servidor 1234</code>	Lanza el servidor escuchando en el puerto local 1234.
make cliente	<code>dotnet build en Cliente/</code>	Compila el proyecto C# generando binarios en <code>obj/</code> .
make abrir-cliente	<code>dotnet run 127.0.0.1 1234</code>	Inicia una instancia interactiva del cliente CLI.
make reporte	<code>tectonic ... reporte.tex</code>	Compila el código fuente LaTeX a PDF.
make abrir-reporte	<code>xdg-open Reporte/reporte.pdf</code>	Abre el visor de documentos PDF del sistema.
make limpiar	<code>rm y dotnet clean</code>	Elimina binarios compilados y archivos temporales.

Cuadro 4: Metas y comandos definidos en el archivo Makefile.

5. Conclusiones

El desarrollo del proyecto permitió aprender de manera práctica los conceptos fundamentales de modelado y programación de sistemas concurrentes y distribuidos:

1. **Eficacia del Protocolo Basado en JSON y Saltos de Línea:** La elección JSON delimitadas por `\n` demostró ser sumamente robusta. Facilitó la interoperabilidad entre dos entornos con modelos distintos: el nivel de C y el nivel de C#.
2. **Control de Concurrencia y Sincronización:** La implementación de exclusión mutua con cerrojos independientes (`mutexUsuarios` y `mutexSalas`) para garantizar la integridad del estado en memoria, permitiendo que múltiples usuarios interactúen simultáneamente sin inducir condiciones de carrera, inconsistencias de punteros ni bloqueos mutuos.
3. **Resolución del Desafío de Framing en TCP:** Se demostró experimentalmente que asumir que una lectura `recv()` equivale a un paquete lógico es un error grave en redes. La implementación del buffer con búsqueda y compactación de memoria resolvió de forma definitiva el manejo de mensajes fragmentados o concatenados.

6. Bibliografía

- [1] djgaven588. *ICC: Cliente simple de IRC implementado en C#*. Repositorio de código abierto en GitHub. Disponible en: <https://github.com/djgaven588/ICC>
- [2] Parth Nan. *FullTCP-Chat-in-C: Servidor de chat multi-cliente en C usando sockets TCP y POSIX Threads*. Repositorio de código abierto en GitHub. Disponible en: <https://github.com/parthnan/FullTCP-Chat-in-C>
- [3] Simple Works. *chat-starter: Plantilla básica para aplicaciones de chat multiusuario*. Repositorio de código abierto en GitHub. Disponible en: <https://github.com/simple-works/chat-starter/>

- [4] Troy D. Hanson. *uthash: Implementación de tablas hash para estructuras en C*. Documentación y código fuente bajo licencia BSD. Disponible en: <https://github.com/troydhanson/uthash>
- [5] Yarthax23. *unix-chat-server: Servidor de chat multihilo con sockets UNIX en C*. Repositorio de código abierto en GitHub. Disponible en: <https://github.com/Yarthax23/unix-chat-server>
- [6] Refactoring Guru. *Catálogo de Patrones de Diseño de Software*. Guía de referencia sobre patrones creacionales, estructurales y de comportamiento en español. Disponible en: <https://refactoring.guru/es/design-patterns/catalog>
- [7] Microsoft Learn. *Patrón Publisher-Subscriber (Publicador-Suscriptor)*. Guía de arquitectura de aplicaciones en Azure y sistemas distribuidos en español. Disponible en: <https://learn.microsoft.com/es-es/azure/architecture/patterns/publisher-subscriber>
- [8] Dave Gamble. *cJSON: Analizador sintáctico ultraligero de JSON en ANSI C*. Repositorio de código abierto en GitHub. Disponible en: <https://github.com/DaveGamble/cJSON>
- [9] Microsoft Corporation. *Programación de red en .NET: Documentación de la clase TcpClient y System.Text.Json*. Microsoft Learn, 2024. Disponible en: <https://learn.microsoft.com/es-es/dotnet/api/system.net.sockets.tcpclient>